

ÉCOLE DE TECHNOLOGIE SUPÉRIEURE
UNIVERSITÉ DU QUÉBEC

RAPPORT DE LABORATOIRE
PRÉSENTÉ À
M. FABIO PETRILLO
DANS LE CADRE DU COURS
ARCHITECTURE LOGICIELLE (LOG430-01)

Laboratoire 9 — Bases de données distribuées et verrous distribués

PAR
Ashley Lester Ian GUEVARRA, GUEA70370101

MONTREAL, LE 2 AVRIL 2026

Table des matières

- [Question 1](#)
- [Question 2](#)
- [Question 3](#)
- [Question 4](#)
- [Question 5](#)
- [Question 6](#)
- [Question 7](#)
- [Section 6 — Environnement de production et CI](#)

Question 1

Quelle est la sortie du terminal que vous obtenez ? Si vous répétez cette commande sur **yugabyte2** et **yugabyte3**, est-ce que la sortie est identique ? Illustrez votre réponse avec des captures d'écran ou des sorties du terminal.

Procédure

Cluster à trois nœuds (**docker compose --profile cluster up -d** dans **yugabyte-db/**). Depuis l'hôte :

```
docker exec yugabyte1 ysqlsh -h yugabyte1 -U yugabyte -c "SELECT * FROM
orders ORDER BY id;"
docker exec yugabyte2 ysqlsh -h yugabyte2 -U yugabyte -c "SELECT * FROM
orders ORDER BY id;"
docker exec yugabyte3 ysqlsh -h yugabyte3 -U yugabyte -c "SELECT * FROM
orders ORDER BY id;"
docker exec python_app python tests/concurrency_test.py --host
http://127.0.0.1:5000 --threads 5 --product 3
```

Ensuite j'ai refait les trois **SELECT** sur yb1, yb2 et yb3.

Réponse

Oui, c'est **identique** sur les trois nœuds. Avant le test, **orders** était vide partout. Après le **concurrency_test** (5 threads, produit 3), le script fait d'abord le pessimiste puis l'optimiste, avec reset du stock entre les deux : au total **quatre** commandes passent (deux par mode — le produit 3 part avec un stock de 2). Les trois requêtes renvoient les **mêmes quatre lignes** (**id**, montants, dates alignés), donc la réplication entre nœuds fonctionne comme prévu.

Sortie terminal — avant le test (**SELECT * FROM orders ORDER BY id** sur yb1, yb2, yb3) :

```
id | user_id | total_amount | payment_link | is_paid | created_at
---+-----+-----+-----+-----+-----
(0 rows)

id | user_id | total_amount | payment_link | is_paid | created_at
---+-----+-----+-----+-----+-----
(0 rows)

id | user_id | total_amount | payment_link | is_paid | created_at
---+-----+-----+-----+-----+-----
(0 rows)
```

Sortie terminal — après le test (même requête sur **yugabyte1**, **yugabyte2**, **yugabyte3**) :

yugabyte1 :

id	user_id	total_amount	payment_link	is_paid	created_at
1	2	5.75		f	2026-04-03 03:37:51.036389+00
2	3	5.75		f	2026-04-03 03:37:51.519155+00
101	1	5.75		f	2026-04-03 03:37:51.077821+00
102	2	5.75		f	2026-04-03 03:37:51.429174+00
(4 rows)					

yugabyte2:

id	user_id	total_amount	payment_link	is_paid	created_at
1	2	5.75		f	2026-04-03 03:37:51.036389+00
2	3	5.75		f	2026-04-03 03:37:51.519155+00
101	1	5.75		f	2026-04-03 03:37:51.077821+00
102	2	5.75		f	2026-04-03 03:37:51.429174+00
(4 rows)					

yugabyte3:

id	user_id	total_amount	payment_link	is_paid	created_at
1	2	5.75		f	2026-04-03 03:37:51.036389+00
2	3	5.75		f	2026-04-03 03:37:51.519155+00
101	1	5.75		f	2026-04-03 03:37:51.077821+00
102	2	5.75		f	2026-04-03 03:37:51.429174+00
(4 rows)					

Test de concurrence (extrait terminal) : pessimiste 2 OK / 3 échecs, moyenne totale **0,315 s** ; optimiste 2 OK / 3 échecs, moyenne totale **0,229 s**.

Question 2

Observez la latence moyenne des deux approches affichée dans la sortie du test. Laquelle a la latence moyenne la plus élevée et pourquoi ? Illustrez votre réponse avec les sorties du terminal.

Procédure

```
docker exec python_app python tests/concurrency_test.py --host
http://127.0.0.1:5000 --threads 20 --product 3
curl -s http://localhost:5001/stocks
```

Produit 3 : stock 2 après reset. Avec 20 threads, 2 commandes passent et 18 finissent en 409 pour chaque mode (normal).

Réponse

Le **pessimiste** a la moyenne totale la plus haute : **0,630 s** vs **0,521 s** pour l'optimiste. En pessimiste, **SELECT ... FOR UPDATE** prend un verrou sur la ligne de stock : les threads s'alignent, donc les succès traînent (**0,464 s** en moyenne vs **0,276 s** en optimiste). Les échecs pessimistes aussi (**0,649 s**), probablement parce qu'ils attendent encore avant de voir que le stock est à zéro. L'optimiste n'a pas cette file au verrou de la même façon ; ses échecs restent dans le même ordre de grandeur mais la moyenne globale sort plus basse ici.

Synthèse des latences (20 threads)

Stratégie	Réussites / échecs	Moyenne succès	Moyenne échecs	Moyenne totale
Pessimiste	2 / 18	0,464 s	0,649 s	0,630 s
Optimiste	2 / 18	0,276 s	0,548 s	0,521 s

Vérification stock (produit 3 = 0)

```
[{"product_id":1,"quantity":1000}, {"product_id":2,"quantity":500},
{"product_id":3,"quantity":0}, {"product_id":4,"quantity":90}]
```

Extrait du code pessimiste dans `yugabyte-db/src/controllers/order_controller.py` :

```
stock = (
    session.query(Stock)
    .filter(Stock.product_id == pid)
    .with_for_update()
    .one_or_none()
)
```

Question 3

Répétez le test avec 5 threads au lieu de 20. Quelle approche a actuellement la latence moyenne la plus élevée et pourquoi ? Illustrez votre réponse avec les sorties du terminal.

Procédure

```
docker exec python_app python tests/concurrency_test.py --host
http://127.0.0.1:5000 --threads 5 --product 3
```

Réponse

Même tendance qu’avec 20 threads : le pessimiste reste au-dessus en moyenne totale (**0,315 s** vs **0,229 s**). Moins de threads = moins de contention, donc les deux chiffres baissent par rapport à la Q2, mais le verrou exclusif du pessimiste coûte encore. Les succès optimistes sont plus rapides en moyenne (**0,16 s** contre **0,279 s** pour les deux succès pessimistes), ce qui tire la moyenne optimiste vers le bas.

Synthèse des latences (5 threads)

Stratégie	Réussites / échecs	Moyenne succès	Moyenne échecs	Moyenne totale
Pessimiste	2 / 3	0,279 s	0,313 s	0,315 s
Optimiste	2 / 3	0,16 s	0,324 s	0,229 s

(Même exécution que la Q1, 2026-04-03 ~03:37.)

Question 4

En utilisant YugabyteDB, quelle stratégie de verrouillage affiche le plus bas taux d'erreurs et la plus basse latence moyenne ? Illustrez votre réponse avec des captures d'écran ou statistiques de l'interface Locust.

Procédure

Reset des stocks (hook dans le locustfile, ou à la main : `curl -X POST http://localhost:5001/stocks/reset`). Puis Locust en headless, 50 users, +5/s, 60 s :

```
docker exec locust locust -f /mnt/locust/locustfile.py --headless -u 50 -r 5 -t 60s --host http://python-app:5000
```

Les chiffres ci-dessous viennent du **résumé imprimé dans le terminal** à la fin du run.

Réponse

YugabyteDB, run du 2026-04-03 (60 s) : le **pessimiste** (`POST /orders/pessimistic`) fait mieux sur les commandes — moins d'échecs et latences moyenne / médiane plus basses que l'optimiste :

Indicateur	Pessimiste	Optimiste
Requêtes	1757	1093
Échecs	841 (47,87 %)	651 (59,56 %)
Latence moyenne (ms)	437	896
Médiane (ms)	150	550
p95 (ms)	1800	3000

Les erreurs sont surtout des **409** (stock / conflit sous charge), ce qui colle au scénario.

Locust — extrait du terminal (fin de run)

```
POST /orders/optimistic      1093 reqs   651 fails (59.56%)   Avg 896 ms   Med 550 ms
POST /orders/pessimistic    1757 reqs   841 fails (47.87%)   Avg 437 ms   Med 150 ms
GET /stocks                  279 reqs     0 fails (0.00%)   Avg 112 ms   Med 72 ms
```

Percentiles

```
POST /orders/optimistic      50% 550 ms ... 95% 3000 ms ... 100% 5400 ms
POST /orders/pessimistic    50% 150 ms ... 95% 1800 ms ... 100% 5400 ms
```


Mon interprétation : avec ce mélange de produits et cette charge, le pessimiste enchaîne les mises à jour de stock de manière plus ordonnée ; l'optimiste se prend plus de conflits / retries, d'où des moyennes et une médiane plus hautes.

Question 5

Est-ce que le taux d'erreur a augmenté lors de l'arrêt du nœud ? Combien de temps a duré le basculement (approximativement) ? Illustrez votre réponse avec des captures d'écran et statistiques de l'interface Locust.

Procédure

```
docker exec locust locust -f /mnt/locust/locustfile.py --headless -u 50 -r 5 --host http://python-app:5000
```

Pendant que Locust tourne : `docker stop yugabyte2`, je surveille les agrégats dans le terminal, puis `docker start yugabyte2`. J'ai arrêté le test avec **Ctrl+C** quand j'avais assez de lignes à comparer.

Réponse

Oui, le pourcentage d'échecs grimpe clairement pendant le run. Ce n'est pas que la panne : sur une charge longue le stock s'épuise aussi (locustfile aléatoire), donc les 409 montent naturellement. En plus, quand un nœud est down, il y a moins de marge et on voit parfois des pics de latence ou des `req/s` qui chutent un moment.

J'ai copié le **premier** bloc d'agrégats utile dans mon terminal et le **dernier** avant Ctrl+C :

- Début (premier extrait) : **~49,5 %** d'échecs au total sur l'agrégat, **~54 %** sur chaque **POST** commande.
- Fin : **~70,6 %** au total ; optimiste **~78,5 %** d'échecs, pessimiste **~77,6 %**. Les **GET /stocks** restent à 0 % d'échec, mais la moyenne monte (vers **383 ms** en fin vs **~493 ms** au début de ce que j'ai gardé).

On voit aussi des max très hauts sur les **POST** (plusieurs secondes) et des phases où le débit baisse ; ça peut coller à la fenêtre où **yugabyte2** était arrêté ou au cluster qui se réorganise.

Durée du basculement : je n'avais pas noté l'heure au deuxième près. En général sur ce genre de lab, ça se compte plutôt entre **~30 s et ~2 min** avant que les chiffres redeviennent « stables », selon la charge. Pour être plus précis la prochaine fois, je noterais l'heure du **stop / start** à côté des lignes Locust.

Extrait terminal — début

POST /orders/optimistic	494 reqs	266 fails (53.85%)	Avg 2185 ms	Med 2100 ms
POST /orders/pessimistic	1194 reqs	649 fails (54.36%)	Avg 692 ms	Med 540 ms
GET /stocks	161 reqs	0 fails (0.00%)	Avg 493 ms	Med 360 ms
Aggregated	1849 reqs	915 fails (49.49%)	Avg 1074 ms	Med 700 ms

Extrait terminal — fin (avant Ctrl+C)

POST /orders/optimistic	1352 reqs	1061 fails (78.48%)	Avg 2267 ms	Med 2100 ms
POST /orders/pessimistic	3579 reqs	2777 fails (77.59%)	Avg 629 ms	Med 470 ms
GET /stocks	506 reqs	0 fails (0.00%)	Avg 383 ms	Med 280 ms
Aggregated	5437 reqs	3838 fails (70.59%)	Avg 1014 ms	Med 590 ms

En résumé : beaucoup de 409 viennent du stock qui fond ; le stop de nœud peut ajouter du stress. Le service ne tombe pas complètement pour autant — les requêtes continuent, ce qui correspond à l'idée de tolérance aux pannes sur un cluster.

Question 6

En utilisant CockroachDB, quelle stratégie de verrouillage affiche le plus bas taux d'erreurs et la plus basse latence ? Illustrez votre réponse avec des captures d'écran ou statistiques de l'interface Locust.

Procédure

- 1. Libérer les ports **5001** et **8089** (partagés avec Yugabyte) : dans `yugabyte-db/`, `docker compose --profile cluster down`.
- 2. Depuis `cockroach-db/` : `docker compose up -d --build` (`.env` à partir de `.env.example` si besoin).
- 3. Réinitialiser les stocks (API sur l'hôte, **port 5001** comme pour Yugabyte) :

```
curl -X POST http://localhost:5001/stocks/reset
```

- 4. Test **headless** 60 s, 50 utilisateurs, spawn 5/s (même scénario que la Q4) :

```
docker compose exec locust locust -f /mnt/locust/locustfile.py --headless -u 50 -r 5 -t 60s --host http://python-app:5000
```

Locust peut quitter avec le code **1** quand il y a beaucoup d'échecs HTTP ; ce n'est pas un bug du setup.

Réponse

CockroachDB, même jour, 60 s, mêmes paramètres que la Q4 : encore le **pessimiste** devant (moins d'échecs, moyennes et percentiles plus bas que l'optimiste sur les **POST**).

Indicateur	Pessimiste	Optimiste
Requêtes	950	394
% échecs	43,79 %	69,54 %
Latence moyenne (ms)	1005	2865
Médiane (ms)	560	1200
p95 (ms)	3400	11000

GET /stocks : 134 reqs, 0 % d'échec, moyenne **763 ms**, médiane **470 ms**.

Locust — extrait terminal (fin de run)

```
POST /orders/optimistic      394 reqs  274 fails (69.54%)  Avg 2865 ms  Med 1200 ms
POST /orders/pessimistic    950 reqs  416 fails (43.79%)  Avg 1005 ms  Med 560 ms
```

GET /stocks	134 reqs	0 fails (0.00%)	Avg 763 ms	Med
470 ms				
Aggregated	1478 reqs	690 fails (46.68%)	Avg 1478 ms	Med
680 ms				

Percentiles (terminal)

POST /orders/optimistic	50% 1200 ms ...	95% 11000 ms ...	100% 17000 ms
POST /orders/pessimistic	50% 560 ms ...	95% 3400 ms ...	100% 7400 ms

Même lecture qu'avec Yugabyte : le pessimiste sérialise les écritures, moins de chaos côté client ; l'optimiste prend plus de 409 et des temps qui explosent une fois la charge installée.

Question 7

Quelle base de données affiche le plus bas taux d'erreurs et la plus basse latence ? Est-ce que c'est YugabyteDB ou CockroachDB ? Illustrez votre réponse avec des captures d'écran ou statistiques de l'interface Locust.

Méthode

Comparer les tableaux **Statistics** des questions **4** et **6** : même durée (**60 s**), mêmes paramètres Locust (**50** users, **5/s**). Les deux essais documentés ici sont sur la **même machine**, à quelques minutes d'intervalle ; une petite différence de charge de fond reste possible.

Synthèse comparative (stratégie **pessimiste**, la plus performante dans les deux cas)

Critère	YugabyteDB (Q4)	CockroachDB (Q6)
% échecs (POST pessimiste)	47,87 %	43,79 %
Latence moyenne (ms)	437	1005
Médiane (ms)	150	560
p95 (ms)	1800	3400

Agrégat Locust (tous les endpoints) : **~46,7 %** d'échecs avec Cockroach (690/1478) contre **~52,4 %** avec Yugabyte sur mes nombres de la Q4 ((841+651)/(1757+1093)).

Réponse

Je ne peux pas répondre « une seule base gagne tout » avec mes chiffres :

- Moins d'**erreurs** (pessimiste seul + total du run) : **CockroachDB** ici.
- Latences** plus basses sur le pessimiste (moyenne, médiane, p95) : **YugabyteDB**.

Donc Cockroach a un peu mieux tenu côté taux de 409 sur ce que j'ai mesuré, mais Yugabyte était plus réactif sur les **POST** pessimistes. Si je devais comparer pour un rapport sérieux, je referais les deux tests à la suite, machine au calme, pour limiter l'écart entre runs.

Section 6 — CI

CI — GitHub Actions

Le workflow est dans `.github/workflows/concurrency-test.yml` : il monte **YugabyteDB**, attend le healthcheck sur le port interne **5000** du conteneur API, puis lance dans `python-app` :

```
python tests/concurrency_test.py --host http://127.0.0.1:5000 --threads 5 --product 3
```

Ça tourne sur le runner `ubuntu-latest` (push / PR sur `main` ou `master`). Pour un vrai pipeline, j'ajouterais un job `deploy` avec `needs: concurrency-test` pour ne déployer que si le test passe.

Note : le premier passage CI est long (pull d'images + démarrage du cluster).